

MOMO Quant

Part 7 — DevOps & Deployment Guide

Version: 1.0 Draft

Deployment Style: Docker Compose first

Production Target: Linux VPS or cloud VM

Primary Goal: Keep deployment reliable, secure, and manageable for a solo developer.

1. DevOps Principle

MOMO Quant should not start with Kubernetes, Kafka, complex CI/CD pipelines, or distributed infrastructure.

That would be overengineering.

The correct first deployment model is:

```
Docker Compose
+
Linux VPS
+
MySQL
+
Redis
+
.NET API
+
Python AI Service
+
React Dashboard
```

This is enough for MVP, backtesting, replay, paper trading, and early private usage.

Only move to Kubernetes or distributed services after the system is stable, profitable, and too large for a single host.

2. Runtime Components

MOMO Quant will run these services:

```
momo-api  
momo-dashboard  
momo-ai  
momo-mysql  
momo-redis  
momo-nginx
```

Optional later:

```
momo-worker  
momo-prometheus  
momo-grafana  
momo-loki  
momo-backup
```

For MVP, the .NET API and worker can live together unless background processing becomes heavy.

3. Recommended Docker Compose Structure

Recommended project structure:

```
momo-quant/  
  docker/  
    nginx/  
      nginx.conf  
    mysql/  
      init/  
    redis/  
    api/  
    ai/  
    dashboard/  
  
  src/  
    backend/  
    frontend/  
    ai/
```

```
deploy/  
  docker-compose.yml  
  docker-compose.override.yml  
  .env.example  
  
scripts/  
  backup-db.sh  
  restore-db.sh  
  deploy.sh  
  logs.sh
```

4. Environment Files

Use environment files for deployment settings.

Example:

```
.env  
.env.development  
.env.production
```

Never commit real `.env` files.

Commit only:

```
.env.example
```

Example `.env.example`:

```
APP_ENV=Production  
APP_URL=https://momoquant.yourdomain.com  
  
MYSQL_DATABASE=momo_quant  
MYSQL_USER=momo_user  
MYSQL_PASSWORD=change-this  
MYSQL_ROOT_PASSWORD=change-this  
  
REDIS_PASSWORD=change-this  
  
JWT_SECRET=change-this-long-secret  
JWT_ISSUER=MOMOQuant
```

```
JWT_AUDIENCE=MOMOQuantDashboard

AI_SERVICE_URL=http://momo-ai:8000

ENCRYPTION_KEY=change-this-32-byte-key

BINANCE_BASE_URL=https://fapi.binance.com
BINANCE_WS_URL=wss://fstream.binance.com
```

Real secrets must never be stored in Git.

5. Docker Compose Services

Initial Docker Compose services:

```
services:
  momo-api
  momo-dashboard
  momo-ai
  momo-mysql
  momo-redis
  momo-nginx
```

5.1 momo-api

Runs the .NET backend.

Responsibilities:

```
REST API
SignalR hubs
Authentication
Bot orchestration
Backtesting
Replay
Paper trading
Reporting
Monitoring
Database access
AI service communication
```

Required environment:

```
ConnectionStrings__Default
Redis__ConnectionString
Jwt__Secret
AiService__BaseUrl
Encryption__Key
```

5.2 momo-dashboard

Runs React dashboard as static files served through Nginx or as a container build output.

Preferred production approach:

```
React build
served by Nginx
```

Do not run Vite dev server in production.

5.3 momo-ai

Runs Python FastAPI AI service.

Responsibilities:

```
Market regime detection
Confidence scoring
Anomaly detection
Trade explanation
Parameter recommendation later
```

Expose internally only.

The public internet should not directly access the AI service.

5.4 momo-mysql

Stores persistent platform data.

Must use Docker volume:

```
momo_mysql_data
```

Do not store MySQL data inside the container filesystem only.

5.5 momo-redis

Used for:

```
runtime cache  
bot state  
temporary replay state  
SignalR scaling later  
background queue support later
```

Redis should be password protected.

5.6 momo-nginx

Reverse proxy for:

```
Dashboard  
API  
SignalR  
TLS termination
```

Public traffic should enter through Nginx only.

6. Networking Design

Use private Docker network:

```
momo-network
```

Only Nginx should expose public ports:

```
80
443
```

Internal services should not be publicly exposed:

```
momo-api
momo-ai
momo-mysql
momo-redis
```

Correct flow:

```
User Browser
  |
  v
Nginx HTTPS
  |
  +--> React Dashboard
  |
  +--> .NET API
        |
        +--> MySQL
        +--> Redis
        +--> Python AI Service
```

7. HTTPS / TLS

All browser-to-server communication must use HTTPS in production.

Use:

```
Nginx
Let's Encrypt
Certbot
```

Important correction:

The API payload itself does not need custom manual encryption if HTTPS is correctly configured.

Correct security model:

HTTPS/TLS protects data in transit.
Database/API key encryption protects secrets at rest.
JWT protects authenticated access.
Role permissions protect dangerous actions.
Audit logs track sensitive operations.

Do not invent custom encryption for every API request. That usually creates bugs and false security.

8. Secrets Management

Secrets include:

JWT secret
Database password
Redis password
Exchange API keys
Exchange API secrets
Encryption key
TLS private keys

Rules:

1. Never commit secrets to Git.
2. Use `.env` locally.
3. Use server environment variables in production.
4. Encrypt exchange API keys before saving to database.
5. Never display API secrets after saving.
6. Rotate secrets if exposed.
7. Use separate testnet and live API keys.
8. Use exchange API permissions carefully.

For live exchange keys:

Enable futures trading only if needed.
Disable withdrawals.
Use IP whitelist if exchange supports it.
Use testnet first.

9. Database Migration Strategy

Use Entity Framework Core migrations.

Rules:

```
Migrations must be committed to Git.  
Production migration must be intentional.  
Do not auto-delete production data.  
Do not run destructive migrations casually.  
Backup before production migration.
```

Recommended flow:

```
Create migration locally  
Test migration locally  
Apply migration to staging/test database  
Backup production  
Apply migration to production  
Verify health
```

10. Backup Strategy

Backups are not optional.

MOMO Quant will store valuable trading history, AI decisions, strategy results, and configuration.

Required backups:

```
MySQL database backup  
.env / deployment secret backup  
uploaded/imported historical data backup if stored outside DB  
Docker Compose config backup
```

10.1 Database Backup Schedule

Recommended:

```
Daily database backup
Weekly full backup
Backup before every production deployment
Backup before every migration
```

Backups should be stored outside the server if possible.

Examples:

```
local encrypted archive
cloud storage
external VPS
private object storage
```

10.2 Backup Script

Create script:

```
scripts/backup-db.sh
```

Responsibilities:

```
Dump MySQL database
Compress dump
Add timestamp
Store in backup folder
Optionally upload to remote storage
Delete old backups after retention period
```

Suggested retention:

```
Daily backups: 14 days
Weekly backups: 8 weeks
Monthly backups: 12 months
```

11. Restore Strategy

Backups are useless if restore is not tested.

Create script:

```
scripts/restore-db.sh
```

Restore process:

```
Stop API and workers
Restore database dump
Run migration check
Start services
Verify dashboard
Verify health endpoint
Verify recent data
```

Test restore at least once before trusting the backup system.

12. Logging Strategy

Logs must be useful, not noisy.

Log categories:

```
API logs
Authentication logs
Trading engine logs
Strategy logs
AI logs
Risk logs
Execution logs
Exchange logs
Monitoring logs
Audit logs
Error logs
```

Use structured logging.

Recommended .NET logging:

```
Serilog  
console sink  
file sink  
database sink later if needed
```

Log format should include:

```
timestamp  
level  
module  
tradingSessionId  
symbol  
strategy  
message  
exception  
correlationId
```

13. Log Retention

Recommended MVP retention:

```
Application logs: 14-30 days  
Error logs: 90 days  
Audit logs: permanent or long-term  
Trading decisions: permanent  
Trades/orders/fills: permanent
```

Do not delete trading records just to save space early.

14. Monitoring Strategy

MVP monitoring should include:

```
API health  
Database health  
Redis health
```

```
AI service health
Exchange REST health
Exchange WebSocket health
Worker status
Bot status
Last market update time
Last AI response time
Last order update time
Error count
Latency
```

Expose backend endpoint:

```
GET /api/v1/monitoring/health
```

Dashboard should display:

```
Healthy
Warning
Critical
Stopped
```

15. Health Checks

Each container should have a health check.

Examples:

```
momo-api      -> /health
momo-ai       -> /health
momo-dashboard -> nginx static response
momo-mysql    -> mysqladmin ping
momo-redis    -> redis-cli ping
```

If a critical service is unhealthy, the bot should not continue trading blindly.

16. Alerts

MVP alert types:

```
Emergency stop triggered
Exchange disconnected
AI service unavailable
Database unavailable
Redis unavailable
WebSocket stale
Daily loss limit reached
Backtest completed
Paper trading stopped unexpectedly
Critical error logged
```

Initial delivery:

```
In-app dashboard alerts
```

Later delivery:

```
Email
Telegram
Discord
SMS
```

Do not overbuild external notifications in MVP.

17. CI/CD Strategy

For MVP, keep CI/CD simple.

Recommended Git workflow:

```
main
develop
feature/*
```

Recommended checks before merge:

```
Backend build
Backend tests
Frontend build
```

```
AI service tests
Docker build
```

Possible CI tool:

```
GitHub Actions
```

Do not create a complex deployment pipeline before the app is stable.

18. Deployment Flow

Recommended manual deployment flow first:

```
Pull latest code
Backup database
Build Docker images
Run migrations
Restart containers
Check health endpoints
Check dashboard
Check logs
Run smoke tests
```

Create script:

```
scripts/deploy.sh
```

The script should not hide important errors.

19. Smoke Tests

After deployment, verify:

```
Dashboard loads
Login works
API health is healthy
Database connection works
Redis connection works
```

```
AI service health works
Symbols load
Strategies load
Bot status loads
Backtest can start in test mode
SignalR connects
```

Do not deploy and assume it worked.

20. Development Environment

Local development should support:

```
.NET API running locally
React dashboard via Vite
Python AI service locally
MySQL in Docker
Redis in Docker
```

Recommended local startup:

```
docker compose up mysql redis
dotnet run
npm run dev
uvicorn app.main:app --reload
```

Later you can run everything in Docker.

21. Production Environment

Production should use:

```
Linux VPS
Docker
Docker Compose
Nginx
HTTPS
MySQL volume
Redis volume
```


Backup folder
Firewall

Minimum VPS for MVP:

2 CPU cores
4 GB RAM
80 GB SSD

Better:

4 CPU cores
8 GB RAM
160 GB SSD

Backtesting can be CPU-heavy. If backtests become slow, optimize later or move heavy backtests to a separate worker.

22. Firewall Rules

Only expose:

80/tcp
443/tcp
22/tcp restricted

Do not expose:

MySQL
Redis
AI service
Internal API ports

SSH should be protected with:

SSH key authentication
No password login

```
Non-root user
Fail2ban optional
```

23. Nginx Routing

Recommended routes:

```
https://yourdomain.com/      -> React dashboard
https://yourdomain.com/api/   -> .NET API
https://yourdomain.com/hubs/  -> SignalR
```

Do not expose:

```
/ai/
```

The Python AI service should remain internal.

24. SignalR Deployment Notes

SignalR over Nginx requires WebSocket support.

Nginx must support:

```
Upgrade header
Connection upgrade
Long timeout for WebSocket
```

If multiple API instances are added later, SignalR may need Redis backplane.

For MVP, one API instance is enough.

25. Data Storage Rules

Persistent data must live in volumes:

```
momo_mysql_data  
momo_redis_data if persistence enabled  
momo_logs  
momo_backups
```

Do not rely on container filesystem for permanent data.

Containers can be destroyed. Volumes must survive.

26. Historical Data Storage

Historical candle data can become large.

Initial storage:

```
MySQL Candles table
```

Later enhancement:

```
compressed parquet files  
object storage  
separate analytics database
```

Do not optimize this too early.

Start with MySQL and proper indexes.

27. Performance Considerations

Performance risks:

```
Large candle queries  
Heavy backtests  
Too many SignalR events  
AI service latency  
Database write volume  
Inefficient reports
```

MVP mitigations:

```
Use pagination
Use indexes
Batch insert candles
Cache dashboard summary
Limit active symbols to top 5
Limit initial timeframes
Run heavy backtests as background jobs
Store only useful realtime updates
```

28. Background Jobs

Use .NET hosted services first.

Jobs:

```
Market data collector
Candle importer
Indicator recalculation
Backtest runner
Replay runner
Paper trading engine
Health checker
Report generator
```

If jobs become complex later, consider:

```
Hangfire
Quartz.NET
Redis queue
Separate worker service
```

Do not add them before necessary.

29. Exchange Connectivity Resilience

Exchange connections will fail. Design for it.

Required behavior:

```
Reconnect WebSocket automatically
Detect stale market data
Log reconnect attempts
Stop trading if data is stale
Validate REST responses
Respect rate limits
Handle API downtime
Handle partial order updates
```

If market data is stale, the bot must not trade.

30. Deployment Safety Rules

Before enabling paper/live engine:

```
Database healthy
Redis healthy
AI service healthy or fallback configured
Exchange connection healthy
Symbols active
Risk profile active
Emergency stop available
Trading mode confirmed
```

For live trading later, also require:

```
Live readiness check passed
API key test passed
Paper trading validation passed
Manual confirmation entered
```

31. Rollback Strategy

Every deployment should have rollback plan.

Minimum rollback:

```
Keep previous Docker image
Keep previous .env
Keep pre-deployment database backup
Document migration rollback if possible
```

If deployment fails:

```
Stop new version
Restore previous image
Restore database if migration damaged data
Verify health
Review logs
```

Do not deploy without a way back.

32. Versioning

Use versioning for:

```
Backend API
Dashboard
AI service
Database migrations
AI models
Strategy logic
Strategy parameters
```

Recommended app version format:

```
v0.1.0
v0.2.0
v1.0.0
```

Strategy versions matter because backtest results depend on the exact strategy code and parameters used.

33. Audit Requirements for DevOps

Audit these actions:

```
Deployment completed
Migration applied
Bot mode changed
Live mode enabled
Risk setting changed
API key added
API key disabled
Emergency stop triggered
Emergency stop cleared
Strategy parameter changed
```

DevOps actions can affect trading behavior. They need traceability.

34. MVP Deployment Scope

Build first:

```
Docker Compose
Nginx reverse proxy
HTTPS
MySQL volume
Redis container
.NET API container
Python AI container
React dashboard container
Health checks
Basic deployment script
Database backup script
Basic logs
```

Delay:

```
Kubernetes
Terraform
Advanced autoscaling
Prometheus/Grafana
Loki
Centralized tracing
Blue-green deployment
Multi-region deployment
Cloud object storage automation
```

Those are useful later, not now.

35. DevOps Implementation Order

Recommended order:

1. Create Dockerfiles
2. Create Docker Compose
3. Configure MySQL and Redis
4. Containerize .NET API
5. Containerize Python AI service
6. Build React dashboard for production
7. Add Nginx reverse proxy
8. Add health checks
9. Add environment variable configuration
10. Add backup script
11. Add restore script
12. Add deployment script
13. Add HTTPS
14. Add basic monitoring dashboard
15. Add CI build checks

36. Critical DevOps Rules

1. Do not expose MySQL publicly.
 2. Do not expose Redis publicly.
 3. Do not expose Python AI service publicly.
 4. Use HTTPS in production.
 5. Do not commit secrets.
 6. Backup before migrations.
 7. Test restore process.
 8. Keep live trading disabled by default.
 9. Use Docker volumes for persistent data.
 10. Monitor stale market data.
 11. Log every deployment-affecting action.
 12. Do not add Kubernetes early.
 13. Do not run Vite dev server in production.
 14. Do not trust a deployment until smoke tests pass.
-

37. Final DevOps Summary

The MOMO Quant deployment should be simple, boring, and reliable.

The goal is not to impress anyone with infrastructure.

The goal is to:

- Run the platform safely.
- Protect data.
- Protect secrets.
- Recover from failure.
- Monitor system health.
- Avoid accidental live trading.
- Make deployments repeatable.

The most important DevOps rule:

- Start simple, but do not be careless.